Bitácora S08 · Saga orquestada y circuit breaker

Alumno: Francisco Javier Pérez Nava

Parte 0. Reproducción del problema

Para comenzar el laboratorio, creé dos cuentas bancarias. La primera fue la cuenta de origen, con un saldo inicial de $10,000, y la segunda fue la cuenta destino, con $5,000.

Después realicé una transferencia de $1,000 y configuré la pausa de 20 segundos entre el cargo y el abono. Durante ese tiempo apagué el servicio de cuentas para provocar una falla y observar qué sucedía con el dinero.

Al volver a encender el servicio y consultar los saldos, observé que la cuenta de origen tenía $9,000, mientras que la cuenta destino seguía con $5,000.

¿Por qué pasó?

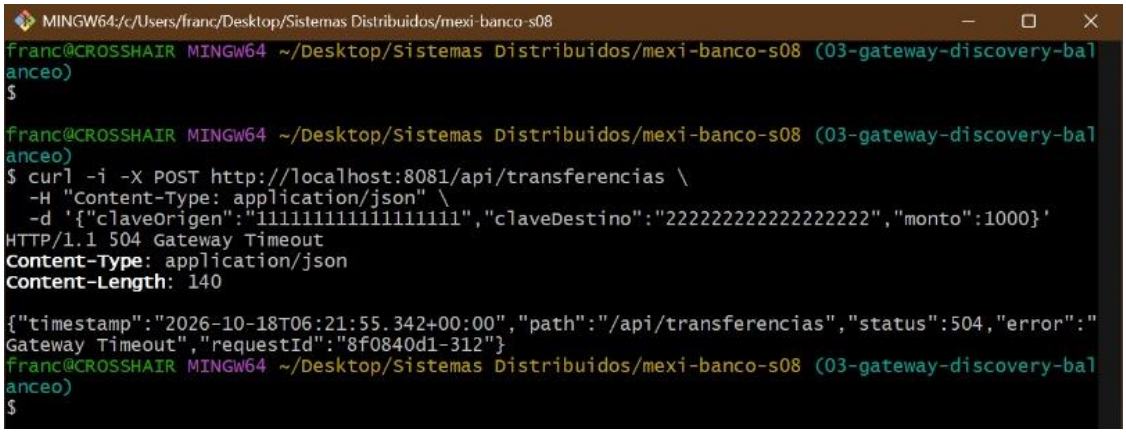
Esto ocurrió porque el dinero se descontó correctamente de la cuenta de origen, pero el servicio falló antes de realizar el abono. Como cada operación se realiza de manera independiente, el dinero no regresó automáticamente a la cuenta original.

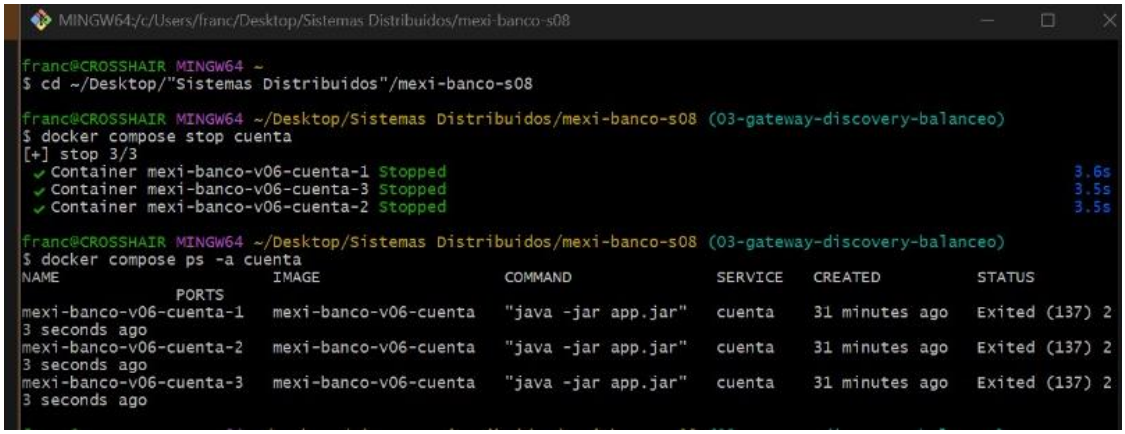
```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -i -X POST http://localhost:8081/api/cuentas \
  -H "Content-Type: application/json" \
  -d '{"clabe":"111111111111111111","titular":"Francisco Perez","saldoInicial":10000}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 5335eaa77175
Content-Type: application/json
Date: Sun, 18 Oct 2026 06:16:31 GMT

{"id":4,"clabe":"111111111111111111","titular":"Francisco Perez","saldo":10000}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -i -X POST http://localhost:8081/api/cuentas \
  -H "Content-Type: application/json" \
  -d '{"clabe":"222222222222222222","titular":"Cuenta Destino","saldoInicial":5000}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: c03c53b57181
Content-Type: application/json
Date: Sun, 18 Oct 2026 06:16:44 GMT

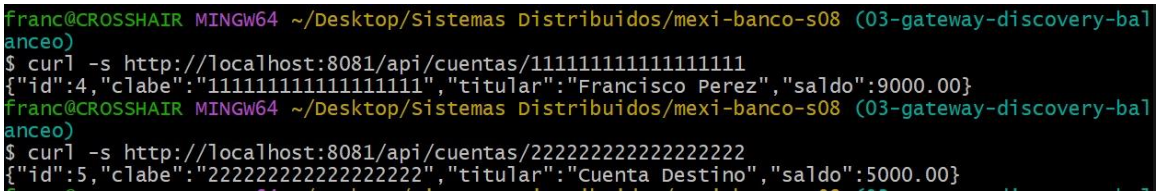
{"id":5,"clabe":"222222222222222222","titular":"Cuenta Destino","saldo":5000}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -s http://localhost:8081/api/cuentas/111111111111111111
{"id":4,"clabe":"111111111111111111","titular":"Francisco Perez","saldo":10000.00}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -s http://localhost:8081/api/cuentas/222222222222222222
{"id":5,"clabe":"222222222222222222","titular":"Cuenta Destino","saldo":5000.00}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$
```

Transferencia, apagado del servicio y error observado.





Saldos posteriores a la falla ($9,000 y $5,000).



Parte 1. Implementación de la saga orquestada

Para solucionar el problema anterior, implementé una saga orquestada en el servicio de transferencias.

La idea es que el servicio TransferenciaService se encargue de controlar los pasos de una transferencia y guardar su estado en la base de datos.

Primero se comprueba que las cuentas existan y sean diferentes. Después se registra la transferencia con el estado INICIADA y se descuenta el dinero de la cuenta de origen, cambiando su estado a CARGADA.

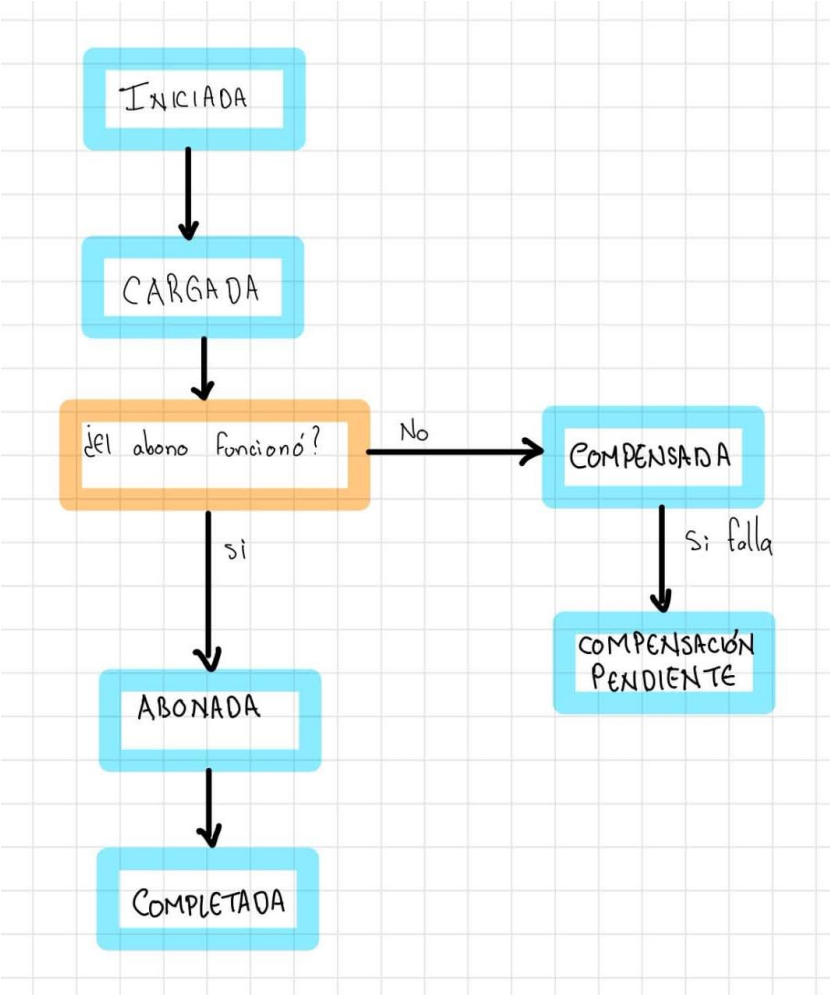
Si el abono a la cuenta destino funciona correctamente, el estado cambia a ABONADA y posteriormente a COMPLETADA, una vez registrados los movimientos.

En cambio, si el abono falla, el sistema intenta devolver el dinero a la cuenta de origen. A este proceso se le llama compensación.

Si logra devolver el dinero, la transferencia queda como COMPENSADA. Si también falla la devolución, se guarda como COMPENSACION_PENDIENTE para indicar que todavía necesita atención.

De esta manera podemos identificar en qué parte del proceso quedó una transferencia y si fue necesario devolver el dinero.

Diagrama de estados de la transferencia



Código de Transferencia.java con el campo estado y sus métodos.

```
@Column(nullable = false)
private String estado = "INICIADA";
```

```
public Transferencia(String claveOrigen, String claveDestino, BigDecimal monto) {
    this.claveOrigen = claveOrigen;
    this.claveDestino = claveDestino;
    this.monto = monto;
    this.fecha = Instant.now();
    this.estado = "INICIADA";
}
```

```java
public String getEstado() {
return estado;
}

public void setEstado(String estado) {
    this.estado = estado;
}
```

Código de TransferenciaService.java con el guardado de estados.

```java
public Transferencia transferir(
            String claveOrigen,
            String claveDestino,
            BigDecimal monto) {

        // PASO 1: Comprobar que las cuentas sean diferentes.
        if (claveOrigen.equals(claveDestino)) {
            throw new ReglaDeNegocioException(
                "La cuenta origen y destino son la misma"
            );
        }

        // PASO 2: Comprobar que las cuentas existan.
        cuentas.consultar(claveOrigen);
        cuentas.consultar(claveDestino);

        // PASO 3: Crear y guardar la transferencia.
        // El estado inicial es INICIADA.
        Transferencia transferencia = transferencias.save(
            new Transferencia(claveOrigen, claveDestino, monto)
        );

        // PASO 4: Descontar dinero de la cuenta origen.
        CuentaRemota origen = cuentas.cargar(claveOrigen, monto);

        // Guardamos que el cargo ya se realizo.
        transferencia.setEstado("CARGADA");
        transferencia = transferencias.save(transferencia);

        // Pausa utilizada para las pruebas de falla.
        pausaDeDemo();

        CuentaRemota destino;

        try {

            // PASO 5: Provocar una falla solamente si
            // configuramos una CLABE para las pruebas.
```

```
            if (claveDestino.equals(clabeFalla)) {
                throw new RuntimeException(
                    "Falla de abono provocada para la prueba"
                );
            }

            // PASO 6: Abonar dinero a la cuenta destino.
            destino = cuentas.abonar(claveDestino, monto);

        } catch (Exception error) {

            // PASO 7: Si falla el abono, intentar
            // devolver el dinero a la cuenta origen.
            try {

                CuentaRemota origenCompensado =
                    cuentas.abonar(claveOrigen, monto);

                // Registrar la devolucion del dinero.
                movimientos.registrar(
                    origenCompensado.clabe(),
                    TipoMovimiento.TRANSFERENCIA_RECIBIDA,
                    monto,
                    origenCompensado.saldo(),
                    "Compensacion por abono fallido"
                );

                // El dinero pudo regresar a origen.
                transferencia.setEstado("COMPENSADA");
                transferencias.save(transferencia);

                throw new ReglaDeNegocioException(
                    "El abono fallo. Se devolvio el dinero al origen."
                );

            } catch (ReglaDeNegocioException errorCompensado) {
                // La compensacion se realizo correctamente.
                throw errorCompensado;

            } catch (Exception errorCompensacion) {

                // No fue posible confirmar la devolucion.
                transferencia.setEstado("COMPENSACION_PENDIENTE");
                transferencias.save(transferencia);

                throw new ReglaDeNegocioException(
                    "Fallo el abono y no se pudo compensar. "
                    + "La transferencia requiere revision."
                );
```

```
            }
        }

        // PASO 8: El abono fue exitoso.
        transferencia.setEstado("ABONADA");
        transferencia = transferencias.save(transferencia);

        // Registrar ambos movimientos.
        MovimientoRemoto cargo = movimientos.registrar(
            origen.clabe(),
            TipoMovimiento.TRANSFERENCIA_ENVIADA,
            monto.negate(),
            origen.saldo(),
            "a " + destino.clabe()
        );

        MovimientoRemoto abono = movimientos.registrar(
            destino.clabe(),
            TipoMovimiento.TRANSFERENCIA_RECIBIDA,
            monto,
            destino.saldo(),
            "de " + origen.clabe()
        );

        // PASO 9: Marcar como completada.
        transferencia.setEstado("COMPLETADA");
        transferencia = transferencias.save(transferencia);

        // Las notificaciones se realizan al final.
        notificaciones.notificar(cargo);
        notificaciones.notificar(abono);

        return transferencia;
    }
```

Cambios del código y para qué sirven

Para implementar la saga orquestada modifiqué principalmente los archivos Transferencia.java y TransferenciaService.java. En el primero agregué un atributo llamado estado, junto con sus métodos para consultarlo y modificarlo, con la finalidad de guardar en la base de datos en qué parte del proceso se encuentra cada transferencia. Después, en TransferenciaService.java, modifiqué el método transferir() para que primero compruebe que las cuentas sean diferentes y que ambas existan. Una vez hecho esto, se registra la transferencia con el estado INICIADA, se descuenta el dinero de la cuenta origen y se actualiza su estado a CARGADA. Si el abono a la cuenta destino se realiza correctamente, el estado cambia a ABONADA, se registran los movimientos y finalmente queda como COMPLETADA. También agregué un bloque try-catch para detectar si ocurre algún error durante el abono y, en ese caso, intentar devolver el dinero a la cuenta origen. Si la devolución funciona, se registra el movimiento y la transferencia queda como

COMPENSADA, pero si también falla, se guarda como COMPENSACION_PENDIENTE para indicar que todavía necesita resolverse. Además, agregué una variable en docker-compose.yml que permite provocar una falla en una CLABE específica y así comprobar que la compensación funcione sin tener que apagar el servicio de cuentas. Con estos cambios, el sistema puede registrar lo que sucede durante la transferencia y tratar de recuperar el dinero cuando el abono falla.

Prueba de compensación funcionando

Para comprobar que la compensación funcionaba, configuré una falla en la cuenta destino y realicé una transferencia de $1,000 desde la cuenta de origen, que tenía $9,000, hacia la cuenta destino, que tenía $5,000. La idea era provocar un error al momento de hacer el abono, pero dejando el servicio de cuentas funcionando para que el dinero pudiera regresar.

Al realizar la transferencia, el sistema descontó los $1,000 de la cuenta de origen, pero el abono falló como se había configurado. En ese momento, la saga realizó la compensación y devolvió el dinero a la cuenta de origen. Después consulté nuevamente los saldos y comprobé que seguían siendo los mismos que antes de la transferencia, es decir, $9,000 en la cuenta de origen y $5,000 en la cuenta destino.

También revisé la base de datos en PostgreSQL y encontré que la transferencia de $1,000, con ID 2, quedó registrada con el estado COMPENSADA. Con esto pude comprobar que, aunque el abono falle, el sistema puede devolver el dinero y guardar un registro de lo que ocurrió.

Configuración de la CLABE de falla y petición de transferencia.

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ docker compose exec transferencia printenv MEXIBANCO_DEMO_FALLAR_ABONO_CLABE
22222222222222222
```

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -i -X POST http://localhost:8081/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"11111111111111111","claveDestino":"22222222222222222","monto":1000}'
HTTP/1.1 422 Unprocessable Entity
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Mon, 19 Oct 2026 00:21:45 GMT

{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"El abono fallo. Se d
evolvio el dinero al origen.","instance":"/transferencias"}
```

Saldos antes y después de la compensación.

Antes

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -s http://localhost:8081/api/cuentas/111111111111111111
{"id":4,"clabe":"11111111111111111","titular":"Francisco Perez","saldo":9000.00}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -s http://localhost:8081/api/cuentas/222222222222222222
{"id":5,"clabe":"22222222222222222","titular":"Cuenta Destino","saldo":5000.00}
```

Después

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -s http://localhost:8081/api/cuentas/111111111111111111
{"id":4,"clabe":"11111111111111111","titular":"Francisco Perez","saldo":9000.00}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -s http://localhost:8081/api/cuentas/222222222222222222
{"id":5,"clabe":"22222222222222222","titular":"Cuenta Destino","saldo":5000.00}
```

Estado COMPENSADA guardado en la base o respuesta de consulta.

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ docker compose exec db printenv POSTGRES_USER
mexibanco
psql (17.11)
Type "help" for help.

transferencia=# \dt
             List of relations
 Schema |     Name      | Type  |   Owner
--------+---------------+-------+-----------
 public | transferencia | table | mexibanco
(1 row)

transferencia=#
transferencia=# SELECT * FROM transferencia ORDER BY id DESC LIMIT 5;
 id |   clave_destino    |    clave_origen    |             fecha             |  monto  |   estado
----+--------------------+--------------------+-------------------------------+---------+------------
  2 | 222222222222222222 | 111111111111111111 | 2026-10-19 00:21:42.907268+00 | 1000.00 | COMPENSADA
  1 | 002180009178069702 | 002180009178069701 | 2026-10-12 04:51:42.932225+00 |  200.00 |
(2 rows)
```

Cuando también falla la compensación

Primero dejé funcionando una sola réplica del servicio de cuentas y configuré una pausa de 20 segundos entre el cargo y el abono. Después inicié una transferencia de $100 y durante esa pausa apagué el servicio de cuentas para provocar que fallara tanto el abono como la devolución.

Al revisar PostgreSQL, observé que la transferencia de $100, con ID 3, había quedado registrada con el estado COMPENSACION_PENDIENTE. Esto significa que el sistema no pudo devolver el dinero en ese momento, pero dejó guardado que todavía existe una operación que necesita resolverse.

Para solucionar esto, se podría agregar un proceso que revise las transferencias pendientes e intente devolver el dinero cuando el servicio de cuentas vuelva a funcionar. También sería importante que la devolución fuera idempotente, es decir, que aunque se intente varias veces, el dinero solamente se devuelva una vez para evitar duplicar la cantidad.

Apagado de cuenta y error de compensación.

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ curl -i --max-time 90 -X POST http://localhost:8081/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"111111111111111111","claveDestino":"222222222222222222","monto":100}'
HTTP/1.1 504 Gateway Timeout
Content-Type: application/json
Content-Length: 140

{"timestamp":"2026-10-19T00:34:21.875+00:00","path":"/api/transferencias","status":504,"error":"Gateway Timeout",
"requestId":"743c9e3f-356"}
```

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-balanceo)
$
docker compose stop cuenta
[+] stop 1/1
 ✔ Container mexi-banco-v06-cuenta-1 Stopped                                    3.3s
```

Estado COMPENSACION_PENDIENTE.

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ docker compose exec db psql -U mexibanco -d transferencia
psql (17.11)
Type "help" for help.

transferencia=# SELECT id, clave_origen, clave_destino, monto, estado
transferencia-# FROM transferencia
transferencia-# ORDER BY id DESC
transferencia-# LIMIT 5;
 id |    clave_origen    |   clave_destino    |  monto  |         estado
----+--------------------+--------------------+---------+------------------------
  3 | 111111111111111111 | 222222222222222222 |  100.00 | COMPENSACION_PENDIENTE
  2 | 111111111111111111 | 222222222222222222 | 1000.00 | COMPENSADA
  1 | 002180009178069701 | 002180009178069702 |  200.00 |
(3 rows)
```

Parte 2. Circuit breaker

Elegí un circuit breaker sencillo implementado en código, sin dependencias adicionales. Sus tres estados son: CERRADO (las llamadas pasan normalmente), ABIERTO (se dejan de hacer llamadas después de varias fallas) y SEMIABIERTO (tras un tiempo de espera, se permite una llamada de prueba). Si la prueba funciona, vuelve a CERRADO; si falla, regresa a ABIERTO.

Clase CircuitBreaker y mensajes de log de los estados.

```
import java.util.function.Supplier;

@Component
public class CuentaCliente {

    private final RestClient http;

    // Estados del circuit breaker.
    private enum Estado {
        CERRADO, ABIERTO, SEMIABIERTO
    }

    // Comienza cerrado para permitir las peticiones.
    private Estado estado = Estado.CERRADO;

    // Cuenta las fallas consecutivas.
    private int fallas = 0;

    // Momento en que se abrio el circuito.
    private long tiempoApertura = 0;

    // Despues de 3 fallas, se abre durante 10 segundos.
    private static final int LIMITE_FALLAS = 3;
    private static final long ESPERA_MS = 10000;

    public CuentaCliente(
            RestClient.Builder builder,
            @Value("${mexibanco.servicios.cuenta}") String urlBase) {

        this.http = ClientesHttp.crear(builder, urlBase, "cuenta");
    }

    // Controla todas las llamadas hacia el servicio cuenta.
    private synchronized <T> T ejecutar(Supplier<T> operacion) {

        // Si esta abierto, revisamos si ya paso el tiempo.
        if (estado == Estado.ABIERTO) {

            long transcurrido =
                System.currentTimeMillis() - tiempoApertura;
```

```
            if (transcurrido < ESPERA_MS) {
                System.out.println(
                    "CIRCUIT BREAKER ABIERTO: solicitud rechazada"
                );

                throw new IllegalStateException(
                    "Servicio cuenta no disponible. Circuit breaker
abierto."
                );
            }

            // Despues de 10 segundos permite una prueba.
            estado = Estado.SEMIABIERTO;
            System.out.println("CIRCUIT BREAKER SEMIABIERTO");
        }

        try {

            // Ejecuta la peticion HTTP.
            T resultado = operacion.get();

            // Si funciona, reinicia el contador.
            fallas = 0;

            if (estado != Estado.CERRADO) {
                estado = Estado.CERRADO;
                System.out.println("CIRCUIT BREAKER CERRADO");
            }

            return resultado;

        } catch (Exception error) {

            // Registra las fallas de comunicacion.
            fallas++;
            System.out.println(
                "CIRCUIT BREAKER: falla " + fallas
            );

            // Abre el circuito al llegar al limite.
            if (fallas >= LIMITE_FALLAS
                    || estado == Estado.SEMIABIERTO) {

                estado = Estado.ABIERTO;
                tiempoApertura = System.currentTimeMillis();

                System.out.println("CIRCUIT BREAKER ABIERTO");
            }
```

```
            throw error;
        }
    }

    // Consulta una cuenta pasando por el circuit breaker.
    public CuentaRemota consultar(String clabe) {
        return ejecutar(() ->
            http.get()
                .uri("/cuentas/{clabe}", clabe)
                .retrieve()
                .body(CuentaRemota.class)
        );
    }

    // Realiza un cargo pasando por el circuit breaker.
    public CuentaRemota cargar(String clabe, BigDecimal monto) {
        return ejecutar(() ->
            http.post()
                .uri("/cuentas/{clabe}/cargos", clabe)
                .body(Map.of("monto", monto))
                .retrieve()
                .body(CuentaRemota.class)
        );
    }

    // Realiza un abono pasando por el circuit breaker.
    public CuentaRemota abonar(String clabe, BigDecimal monto) {
        return ejecutar(() ->
            http.post()
                .uri("/cuentas/{clabe}/abonos", clabe)
                .body(Map.of("monto", monto))
                .retrieve()
                .body(CuentaRemota.class)
        );
    }
}
```

Para implementar el circuit breaker modifiqué el archivo CuentaCliente.java, que es el encargado de comunicarse con el servicio de cuentas. Decidí hacerlo directamente en el código para no tener que agregar librerías adicionales. Primero definí los tres estados del circuito: CERRADO, ABIERTO y SEMIABIERTO. Después agregué un contador para registrar las fallas consecutivas y una variable para guardar el momento en que se abre el circuito. También configuré un límite de tres fallas y un tiempo de espera de 10 segundos. Para controlar todo esto, agregué un método llamado ejecutar(), por el cual ahora pasan las operaciones de consultar cuentas, realizar cargos y hacer abonos. Cuando el circuito está CERRADO, las solicitudes se realizan normalmente y, si alguna falla, aumenta el contador. Al llegar a tres fallas consecutivas, cambia a ABIERTO y rechaza nuevas solicitudes sin intentar comunicarse con el servicio de cuentas. Después del tiempo de espera, pasa a SEMIABIERTO y permite una llamada de prueba para comprobar si el servicio ya funciona.

Si la llamada es exitosa, el contador vuelve a cero y el circuito regresa a CERRADO, pero si falla, vuelve a abrirse. Además, agregué mensajes en los logs para poder observar las fallas, los cambios de estado y las solicitudes rechazadas durante las pruebas. Con esto, el sistema evita seguir enviando peticiones a un servicio que está fallando y permite recuperar la comunicación cuando vuelve a estar disponible.

Estados

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)stamp":"2026-10-19T01:15:13.331+00:00","status":500,"error":"Internal Server Error","path"
$ curl -i --max-time 15 -X POST http://localhost:8081/api/transferencias   -H "Content-Type: app
lication/json"   -d '{"claveOrigen":"111111111111111111","claveDestino":"222222222222222222","mo
nto":1}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 01:15:20 GMT

{"timestamp":"2026-10-19T01:15:20.227+00:00","status":500,"error":"Internal Server Error","path"
:"/transferencias"}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -i --max-time 15 -X POST http://localhost:8081/api/transferencias   -H "Content-Type: app
lication/json"   -d '{"claveOrigen":"111111111111111111","claveDestino":"222222222222222222","mo
nto":1}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 01:15:22 GMT

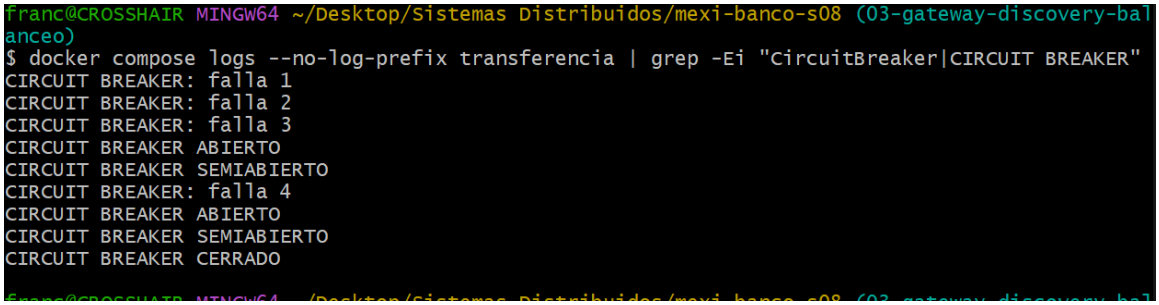
{"timestamp":"2026-10-19T01:15:22.822+00:00","status":500,"error":"Internal Server Error","path"
:"/transferencias"}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -i --max-time 15 -X POST http://localhost:8081/api/transferencias   -H "Content-Type: app
lication/json"   -d '{"claveOrigen":"111111111111111111","claveDestino":"222222222222222222","mo
nto":1}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 01:15:58 GMT

{"timestamp":"2026-10-19T01:15:58.477+00:00","status":500,"error":"Internal Server Error","path"
:"/transferencias"}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ docker compose start cuenta
[+] start 3/3
 ✔ Container mexi-banco-v06-db-1        Healthy                                          0.5s
 ✔ Container mexi-banco-v06-discovery-1 Healthy                                          0.5s
 ✔ Container mexi-banco-v06-cuenta-1    Started                                          0.3s

franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ docker compose ps cuenta
NAME                    IMAGE
    COMMAND               SERVICE   CREATED        STATUS                    PORTS
mexi-banco-v06-cuenta-1   sha256:347f288cda93ec367e7f6919bd6d5c988725e0a567d3e0a1aa834c95adb7d0c
5   "java -jar app.jar"   cuenta    19 hours ago   Up 36 seconds (healthy)   8080/tcp
```

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s08 (03-gateway-discovery-bal
anceo)
$ curl -i --max-time 15 -X POST http://localhost:8081/api/transferencias   -H "Content-Type: app
lication/json"   -d '{"claveOrigen":"111111111111111111","claveDestino":"222222222222222222","mo
nto":1}'
HTTP/1.1 422 Unprocessable Entity
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Mon, 19 Oct 2026 01:16:54 GMT

{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"El abono fallo. Se d
evolvio el dinero al origen.","instance":"/transferencias"}
```



Lo que la saga no resuelve: aislamiento

Aunque la saga permite devolver el dinero cuando una transferencia falla, eso no significa que los cambios no se puedan ver mientras se realiza la operación. Por ejemplo, si una cuenta tiene $10,000 y se realiza una transferencia de $1,000, después del cargo aparecerá un saldo de $9,000, aunque todavía no se haya completado el abono a la otra cuenta.

Si en ese momento alguien consulta el saldo, verá que tiene menos dinero y no sabrá si la transferencia terminará correctamente o si después se realizará una devolución. Esto sucede porque cada servicio realiza sus operaciones por separado y los cambios se van guardando conforme ocurren.

Por eso, aunque la saga ayuda a solucionar los problemas cuando falla una transferencia, no ofrece el mismo aislamiento que una transacción tradicional, donde se puede controlar que los cambios no sean visibles hasta que la operación se confirme.